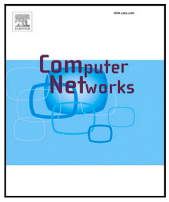
Computer Networks 247 (2024) 110457

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet



DCCS: A dual congestion control signals based TCP for datacenter networks

Yifei Lu *, Xu Ma, Changjiang Cui

School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, 210094, China

ARTICLE INFO

Keywords:
Congestion control
ECN
Delay
Fairness
Datacenter networks

ABSTRACT

As cloud computing advances, datacenter networks (DCNs) face unprecedented pressure due to the increasing prevalence of high bandwidth and low delay applications. Current reliable transmission protocols such as TCP harness packet loss, network delay, and Explicit Congestion Notification (ECN) for congestion control, aiming to maintain low latency and high throughput. We investigate the efficacy of ECN-based protocols against incast congestion in one-hop networks and delay-based protocols for maintaining low end-to-end delay in multi-hop networks, at the expense of throughput fairness. Thus, we introduce a novel TCP protocol, Dual Congestion Control Signals (DCCS), merging ECN and delay signals for a broader congestion control mechanism. DCCS effectively handles burst traffic, prevents packet loss, ensures low end-to-end delay, and resolves throughput unfairness by determining a precise minimum round-trip time (RTT) for each flow through a drain signal. Simulations on ns-3 confirm that DCCS surpasses ECN-based (i.e., DCTCP), delay-based (i.e., DC-Vegas and Swift), and hybrid-based protocols (i.e., EAR) in fat-tree topology under realistic workload, cutting down Flow Completion Time (FCT) by 3.8%–20.5%. This substantiates DCCS's competency in providing high-quality service for DCNs.

1. Introduction

Recent years have witnessed rapid growth in datacenter networks (DCNs) in terms of size and link speed. DCNs play a crucial role in providing quality of service (QoS) for various applications and services, including web search, social networking, data mining, cloud computing, data storage, and more. These applications and services often possess varying requirements, including low latency, high throughput, burst tolerance, and sometimes, both. Moreover, certain applications may trigger simultaneous, large-scale data requests. This phenomenon, characterized by a sudden and high fan-in network traffic within a brief period, culminates in significant congestion of network resources. This is referred to as the incast congestion. To cater to these requirements and issues, numerous recent studies [1–5] have focused on improving TCP congestion control algorithms by leveraging explicit congestion feedback from switches [4,6,7], utilizing Explicit Congestion Notification (ECN) [1,8], and gauging delays [2,5,9–11]. Notably, all of these network congestion control protocols heavily rely on the precision and granularity of congestion feedback.

Nowadays, congestion control algorithms for DCNs primarily rely on three commonly used congestion signals: packet loss, ECN, and network delay. Traditional TCP variants like Reno [12], NewReno [13], and Cubic [14] employ packet loss signals to manage congestion in conventional Wide Area Networks (WANs). However, these algorithms fail to effectively address incast congestion issues specific to DCNs. In contrast, DCTCP [1] and DCQCN [15] utilize ECN to provide multi-bit feedback through switches or routers, mitigating the incast congestion. This ECN-based protocol marks packets when the current switch queue length exceeds a predefined threshold, allowing senders to respond to congestion by reducing their congestion windows proportionally to the number of marked packets. However, the ECN-based protocol performs poorly in networks with multiple bottlenecks, and both the packet loss and ECN congestion signals cannot accurately reflect the congestion level. Consequently, they are considered coarse-grained congestion signals.

Additionally, TCP Vegas [16] and TCP FAST [17] are among the protocols that utilize delay as the primary congestion signal [18], in contrast to traditional TCP variants that rely on packet loss. While these delay-based TCP protocols can maintain low delays, they tend to exhibit suboptimal performance during burst traffic scenarios in DCNs. This is attributed to potential overestimation of delay caused by ACK compression and network jitter, leading to underutilization of the network link. Moreover, accurately measuring end-to-end delay poses significant challenges in DCNs, given their exceptionally high bandwidth (up to 40 Gbps) and remarkably low latency (a few hundred microseconds) [19]. Although techniques like DX [9] and Timely [2] can leverage hardware and software to precisely estimate delay, these solutions are expensive and not widely accessible. Considering these

* Corresponding author.
E-mail address: luyifei@njust.edu.cn (Y. Lu).

https://doi.org/10.1016/j.comnet.2024.110457
Received 1 September 2023; Received in revised form 6 January 2024; Accepted 22 April 2024
Available online 24 April 2024

challenges, a new delay-based TCP protocol named DC-Vegas [10] is proposed, which combines features from TCP Vegas and DCTCP. However, our analysis reveals that DC-Vegas struggles to address the TCP incast issue in large-scale network topologies.

The deployment of both ECN-based and delay-based congestion control protocols is prevalent in DCNs, prompting the question of which congestion signal is better suited in such networks. However, a consensus on the optimal congestion signal has not been reached thus far [20]. Consequently, the identification and utilization of more precise and fine-grained congestion signals have emerged as critical considerations within the field of TCP congestion control for DCNs.

In this paper, we empirically examine ECN-based and delay-based protocols, focusing on their strengths and weaknesses regarding throughput, convergence, and fairness across varied network topologies. Our findings reveal that the ECN-based protocol effectively prevents packet loss during burst traffic in single-rack topologies, while the delay-based protocol better controls end-to-end delay in fat-tree topologies. However, the ECN-based protocol exhibits slower convergence towards equilibrium, whereas the delay-based protocol converges rapidly but faces fairness issues due to inaccurate minimal RTT estimations.

Based on these insights, we aim to strike a balance between ECN and delay congestion signals and find a more accurate fine-grained congestion signal to achieve high performance, low delay, fast convergence, and fairness. Therefore, we propose a dual congestion control signals protocol for TCP in DCNs, named DCCS. DCCS integrates an ECN-based signal, effective in preventing burst packet loss in single-rack topology, with a delay-based signal, adept at maintaining low end-to-end delays in large-scale topology. Moreover, the EAD (ECN And Delay) congestion signal is proposed to address severe congestion scenarios (i.e., multi-bottleneck congestion) by significantly reducing transmission rates for all concurrent flows. To enhance DCCS's fairness, we introduce a drain signal for precise minimum RTT estimation.

This paper primarily addresses intra-DCNs and presents the following key contributions as follows:

1. A comprehensive performance evaluation of ECN-based and delay-based protocols across diverse scenarios. This analysis highlights the inherent strengths and weaknesses of each protocol, pinpointing the root cause of unfairness in delay-based protocols: inaccurate minimal RTT estimations for active flows.
2. Proposal of DCCS, an algorithm combining ECN and delay signals to manage burst congestion using ECN while minimizing end-to-end delay with delay signals. In cases of severe congestion, DCCS employs an aggressive EAD signal, resolving this issue by combining ECN and delay mechanisms. Additionally, the introduction of a drain signal ensures accurate minimum RTT estimation, aiding DCCS in achieving fairness. To our knowledge, this is the first paper to address the unfairness issue of delay-based protocols in DCNs.
3. Evaluation of DCCS using the ns-3 simulator, confirming its effectiveness in maintaining robust throughput, low end-to-end delay, rapid convergence, fairness, and stability.

The rest of the paper is organized as follows. Section 2 provides a review of the related literature. We elaborate on the design motivation in Section 3. Section 4 presents our basic idea and design details of the DCCS algorithm. Extensive simulations, designed to validate the performance of DCCS, are conducted in Section 5. Finally, Section 6 offers concluding remarks on this paper.

2. Related work

The efficacy of network congestion control is fundamentally contingent upon the effective utilization of congestion signals. Existing protocols according to congestion signal can generally be classified into five categories: loss-based signal, explicit in-network signal, delay-based signal, hybrid-based signal, and learning-based signal.

Protocols based on packet loss. Packet loss protocols, which serve as the traditional methods for Internet congestion control, identify network congestion by a timeout or through three duplicate ACKs prompted by packet loss. For example, CUBIC [21] is a loss-based TCP variant widely utilized in Linux. However, due to the distinct network characteristics between the Internet and DCNs, the loss-based method proves ineffective within DCNs.

Some efforts have been made to address these problems. These include the use of a single duplicate ACK, disabling TCP slow start phase [22], and reducing the retransmission TCP timeouts value [23]. The CP scheme [3] reduces packet payloads but not packet headers, enabling rapid loss notifications. Up to a certain extent, these studies can potentially relieve the TCP incast problem.

However, the packet loss is a coarse-grained congestion signal, since it can only convey a single bit of congestion information to distinguish congestion presence or absence without capturing the extent of network congestion.

Protocols based on explicit in-network signal. Some congestion control protocols necessitate in-network support. In traditional network, XCP [24] adjusts the send rate properly using a mechanism capable of providing detailed feedback about network congestion. In DCNs, studies [4,25] allocate each flow a reasonable bandwidth using in-network feedback from switches to fully utilize network bandwidth. FCP [26] and D^3 [6] send a request detailing their bandwidth requirements from senders to the network switches which can satisfy as many requirements for flows as possible in a greedy manner. HPCC [27] leverages INT (in-network telemetry) to obtain fine-grained network load information from switches and then precisely adjusts the congestion window. Bolt [28] employs programmable switches to generate custom control signals to provide fine-grained and precise telemetry and measure minimal control loop delay. Thus it can gain a ultra-low latency at very high line.

Nowdays, ECN-based congestion signal have garnered considerable attention due to their facile deployment within DCNs. DCTCP [1], a pioneer of congestion control protocols in DCNs, exploits the switches' ECN mechanism to mark the incoming packets when the queuing length is larger than a threshold. The receivers will feedback these marked packets with corresponding ACKs. The senders calculate the congestion extent according to the ratio of the received marked ACKs and regulate their congestion window. Thus it can suppress the queue buildups and packet drops leading to low end-to-end latency.

Following DCTCP, a multitude of ECN-based protocols have been proposed. Some works extend the ECN scheme to accommodate deadline-restrictive applications within DCNs, for example, D^2TCP [29], DAP [30], DTP [31], and A^3DCT [5]. Another thread of research aims to augment DCTCP by setting an appropriate ECN marking threshold [8,32,33]. Moreover, ECN marking control schemes tend to be overly aggressive and are prone to overlooking transient congestion. The ABE protocol [34] combines ECN and AQM to enable TCP senders to back off less aggressively. CEMD [35] demonstrates that mis-marking ECN by DCTCP can trigger sender overreaction. It aims to identify and exclude instances of transient queue occupancy.

An ECN-based congestion signal, encompassing only a one-bit ECN flag, is also considered coarse-grained and fails to accurately represent the level of congestion. Furthermore, ECN technology is not universally supported by datacenter hardware.

Protocols based on delay. Transmission delay, such as Round Trip Time (RTT), serves as another traditional congestion control signal for the Internet. For example, TCP Vegas [16] utilizes the difference between base RTT and current RTT to adjust the congestion window but it will be effected by the presence of reverse traffic resulting in not grabbing its bandwidth share [36]. Other algorithms that employ variations of this method include FAST TCP [17] and TCP LoLa [37]. However, delay-based protocols may not be suitable for deployment in DCNs characterized by high bandwidth and low latency. A notable drawback of TCP Vegas is its reliance on accurately measuring RTT in

DCNs, because RTT is very small and may be too sensitive to detect accurately.

Recently, some researchers have attempted to implement the delay-based signal within the DCN environment. For instance, TIMELY [2] employs the rate of RTT variation to predict the congestion and accordingly adjusts the sending rates. Furthermore, it harnesses NIC hardware to facilitate microsecond-accurate RTT measurements. DX [9], another latency-based TCP algorithm for DCNs, uses end-to-end latency as a basis for adjusting the congestion window. It relies on specific NIC hardware with high-quality timestamps. DC-Vegas [10] leverages the ECN-like scheme to design a delay-based TCP algorithm for datacenter congestion control. The estimated queue length is compared with a threshold at the sender to denote the level of congestion. FAMD [11] combines the delay signal, ECN marking scheme, and the characteristics of flows to design a new delay-based TCP protocol aimed at mitigating incast in the DCN environment. GCC [38], which is designed for RTC, uses the end-to-end one-way delay variation from the sender to the destination to infer congestion.

Swift [39] proposed by Google leverages the NIC hardware timestamp feature to divide network congestion into two categories: endpoint congestion and fabric congestion, each corresponding to endpoint delay and fabric delay respectively. By comparing these with the predetermined target delay, Swift is able to measure the extent of congestion. Consequently, this allows for the precise computation of endpoint congestion window and fabric congestion window, which are then employed to modulate the transmission rate.

From the aforementioned discussion, it is clear that an accurate RTT estimation method is pivotal for these delay-based algorithms. However, their inherent sensitivity to RTT variations renders them less competitive in comparison to non-delay-based algorithms.

Protocols based on hybrid congestion signal. Hybrid congestion protocols mix multiple congestion signals to detect network congestion.

In traditional network, some studies have integrated both loss-based and delay-based congestion signals. TCP Africa [40], TCP Compound [41], and TCP YeAH [42] employ a combination of loss-based and delay-based congestion signals for networks with high Bandwidth Delay Product (BDP). Nimbus in [43] utilizes an elasticity detector to switch between a delay-control algorithm and a TCP-competitive algorithm at the sender side.

Both ECN-based and delay-based congestion protocols have been implemented in DCNs. However, while ECN proves more effective in preventing packet loss, RTT is more adept at regulating end-to-end queuing delay. EAR [44] is a proposed methodology that integrates RTT and ECN as indicators of network congestion. It independently computes the magnitude of congestion for both RTT and ECN, then adds up these values to formulate a composite congestion indicator. This synthesized indicator is subsequently used by EAR to manage the congestion window. However, it is noteworthy that EAR has only been substantiated with initial results. BBR [45] proposed by Google adopts RTT and bandwidth as primary congestion signals, striving to achieve high throughput with minimal latency. To attain both fast convergence and TCP-friendliness within a high-speed datacenter network, FFC is introduced in [46]. It uses ECN to detect congestion to prevent throughput loss, subsequently employing the delay signal to estimate the level of congestion and adjust the window accordingly.

Despite several years of research on hybrid signals-based protocols, they are not yet mature enough and lack of consideration for different network scenarios.

Protocols based on machine learning approaches. With the development of artificial intelligence, there has been a gradual increase in congestion control protocols based on machine learning approaches.

Remy [47] employs an offline-trained machine learning model to dynamically allocate congestion window sizes based on the latest TCP session's network conditions. PCC [48], a Performance-oriented Congestion Control method, learns in real-time by assessing the impact of controller actions on performance through gradient ascent. This enables adaptive adjustments in the sending rate to maximize utility. Both Remy and PCC perceive the network as a black box, lacking automatic customization based on experienced network behavior. PCC Vivace [49], building on the high-level architecture of PCC, utilizes online machine learning optimization via gradient ascent to introduce a novel rate-control protocol. NeuRoc [50] utilizes online change point detection and deep reinforcement learning techniques to derive an optimal congestion control policy, enabling TCP to maximize bandwidth usage while minimizing latency. Another approach [51] constructs a machine learning model to discern the most effective algorithm for specific conditions, fine-tuning the selection according to application-layer requirements. The Ref. [52] provides a comprehensive summary and comparison of learning-based congestion control approaches, emphasizing reinforcement learning as a pivotal trend among these algorithms.

Machine learning-based congestion control protocols are in the early stages of development. As learning algorithms progress, especially the application of large models, this research domain is poised to face a blend of opportunities and challenges.

This paper aims to fully explore the advantages of ECN and delay signals, and provide more refined congestion control in different network scenarios.

3. Motivation

3.1. Comprehensive performance analysis

To discern the performance discrepancy between ECN-based and delay-based congestion control protocols under varying circumstances, we employ DCTCP and DC-Vegas as representatives of ECN-based and delay-based TCP protocols respectively. We analyze their performance in terms of flow competition time (FCT), fairness, and convergence.

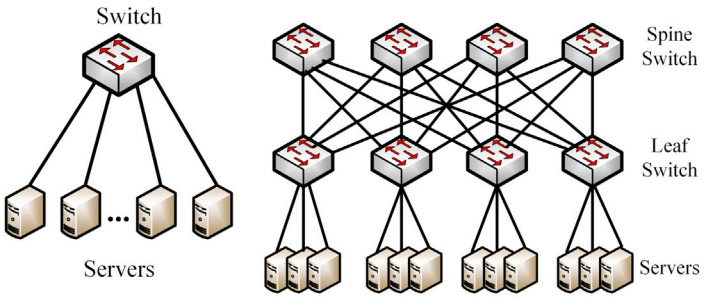
In the following experiments, we assign a capacity of 10 Gbps and a delay of 10 μs to each link in the topology. Additionally, the port buffer size is set with 250 packets. To ensure consistency with prior research, we adopt the configurations recommended in [1,10] for the DCTCP and DC-Vegas algorithms, respectively. Specifically, we chose $K_{\mathrm{ECN}} = 50$ for DCTCP and $K_{\mathrm{dcv}} = 16$ for DC-Vegas. Furthermore, the maximum segment size (MSS) is established as 1460 bytes, and the minimal retransmission timeout (RTO) is set to 5 ms.

Scenario 1. FCT under three different network topologies. In this scenario, we construct three network topologies: a single-rack topology, a leaf-spine topology and an 8-pod fat-tree topology, as depicted in Fig. 1. Finally, across these three topologies, we randomly generate 100, 200, and 800 flows within 5 ms, 5 ms, and 10 ms, respectively, according to the web search workload [1].

We measure the FCT under three network topologies, and the results are shown in Fig. 2. As seen in Fig. 2(a), DCTCP's FCT outperforms that of DC-Vegas in the single-rack topology, whereas, in the fat-tree topology, the opposite is true, as depicted in Fig. 2(c). Moreover, in the leaf-spine topology, Fig. 2(b) presents that DCTCP's performance is similar to that of DC-Vegas. To probe deeper into the reasons behind these results, we examine the instantaneous queue length in the single-rack topology. The results, shown in Fig. 2(d), reveal that the queue length of DC-Vegas is substantially larger than that of DCTCP, which corroborates the aforementioned observations.

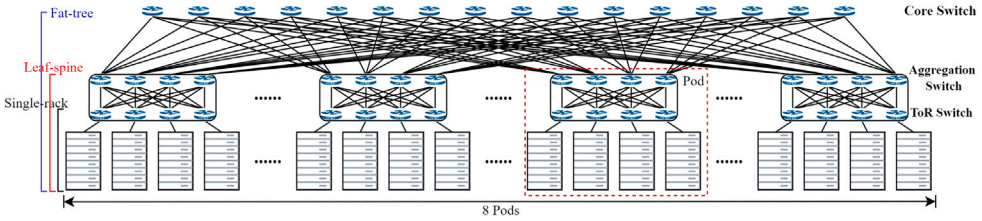
From these results, it is evident that DCTCP outperforms DC-Vegas in the single-rack topology, whereas DC-Vegas exhibits superior performance in the fat-tree topology. This can be attributed to the ECN-based method's ability to react promptly to congestion and maintain the queue length around the threshold.

Scenario 2. Delay under the multi-bottleneck topology. In this scenario, we construct a multi-bottleneck topology as depicted in Fig. 3. There are three groups of flows: Traffic 1 (TR 1) consisting of 50 flows from S1 to R1, Traffic 2 (TR 2) consisting of 30 flows from S2 to R2, and Traffic 3 (TR 3) from S3 to R1 with a variable number of flows. Thus,



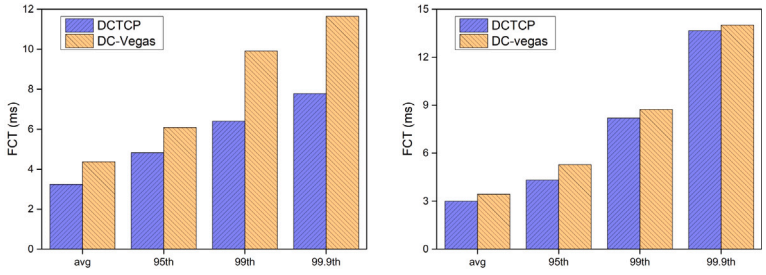
(a) Single-rack network topology

(b) Leaf-spine network topology



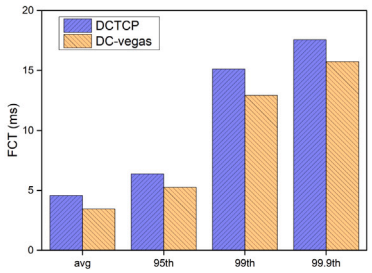
(c) Fat-tree network topology with 8 pods

Fig. 1. Three classic network topologies.

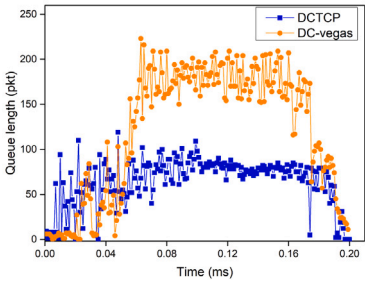


(a) FCT in the single-rack topology

(b) FCT in the leaf-spine topology



(c) FCT in the fat-tree topology



(d) Queue length in single-rack topology

Fig. 2. FCT and queue length under different network topologies.

two network bottlenecks exist, namely B1 and B2. In this experiment, all flows are generated within 1 ms following a Poisson process and set to a size of 5 MB.

We observe the instantaneous queuing delay of B1 and B2 with the varying number of S3. The results are shown in Fig. 4. It can be noted that, from Figs. 4(b), 4(d), 4(f), as the number of S3 increases, the queuing delay of B2 for DC-Vegas is increased quickly from 80 μs to about 270 μs. In contrast, DCTCP's queuing delay remains around 220 μs, though its oscillation becomes larger. However, Figs. 4(a), 4(c), 4(e) depict that the queuing delay of B1 for both DCTCP and DC-Vegas maintains stability at approximately 37 μs and 15 μs, respectively. Moreover, DC-Vegas gains a smaller FCT than DCTCP.

In summary, DC-Vegas outperforms DCTCP in maintaining a low end-to-end delay in a relatively stable network environment under a multi-bottleneck network. However, in a single-rack scenario with burst traffic, DCTCP exhibits superior performance.

Scenario 3. Fairness under the multi-bottleneck topology. In this part, we also construct the network topology illustrated in Fig. 3 to assess the fairness of DCTCP and DC-Vegas. The simulation is divided into two parts: (1) The synchronous simulation. TR 1, TR 2, and TR

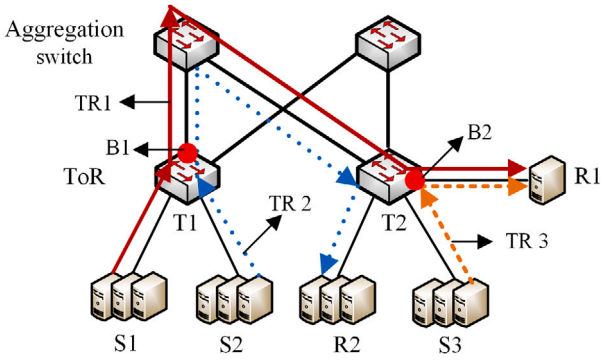
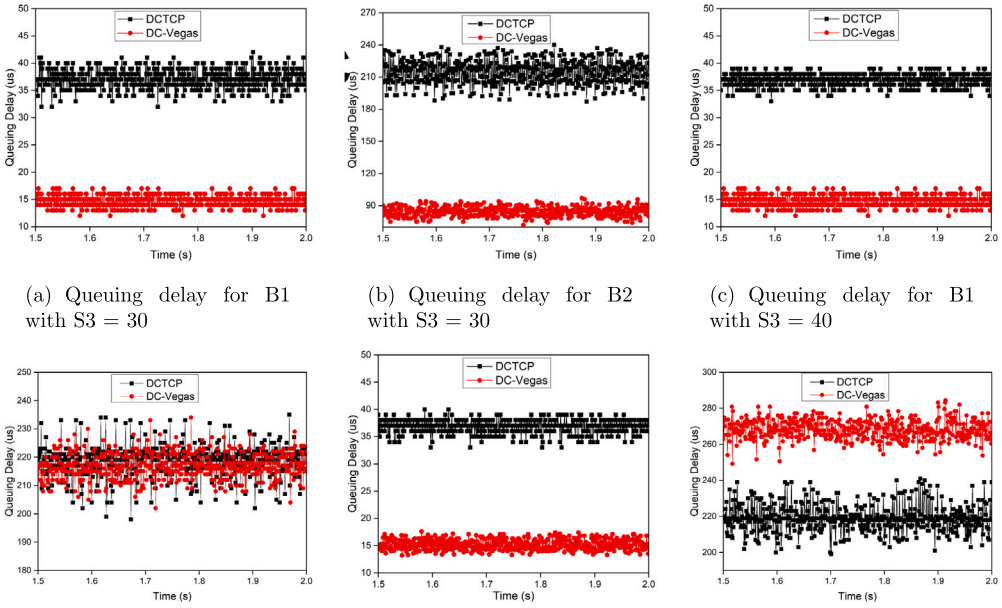


Fig. 3. Multi-bottleneck topology.



(a) Queuing delay for B1 with S3 = 30

(b) Queuing delay for B2 with S3 = 30

(c) Queuing delay for B1 with S3 = 40

(d) Queuing delay for B2 with S3 = 40

(e) Queuing delay for B1 with S3 = 50

(f) Queuing delay for B2 with S3 = 50

Fig. 4. Queuing delay for multi-bottleneck network topology.

Table 1
The fairness index for DCTCP and DC-Vegas.

Protocols	Synchronous		Asynchronous	
	DCTCP	DC-Vegas	DCTCP	DC-Vegas
Fairness (TR 1)	0.993	0.995	0.989	0.672
Fairness (TR 2)	0.991	0.989	0.992	0.771
Fairness (TR 3)	0.995	0.990	0.991	0.724

3 start to run at the same time. (2) Asynchronous simulation, where TR 1, TR 2, and TR 3 begin at staggered intervals of 20 ms. Once the network achieves convergence, we separately calculate the Jain's fairness index [53] for TR 1, TR 2, and TR 3.

The results are displayed in Table 1. In the synchronous simulation, the fairness index for both DCTCP and DC-Vegas is close to 1, indicating commendable fairness. But in the asynchronous simulation, DCTCP can keep good fairness while DC-Vegas exhibits a significantly lower fairness index. The reason for the unfairness of DC-Vegas is that sender may incorrectly estimate a minimal RTT which can lead to an unfair sending rate. We will delve into the reasons behind this in the following section.

Scenario 4. Flow Convergence. In this scenario, for convenience, we use a simple topology like the single-rack topology shown in Fig. 1(a). To observe the convergence of these two protocols, we first set the number of the senders to 2 for simplicity. The flow 1 initiates first and, after a delay of 0.2 s, flow 2 begins to send data. These two flows continue to send packets for 1 s and the results are presented in Fig. 5.

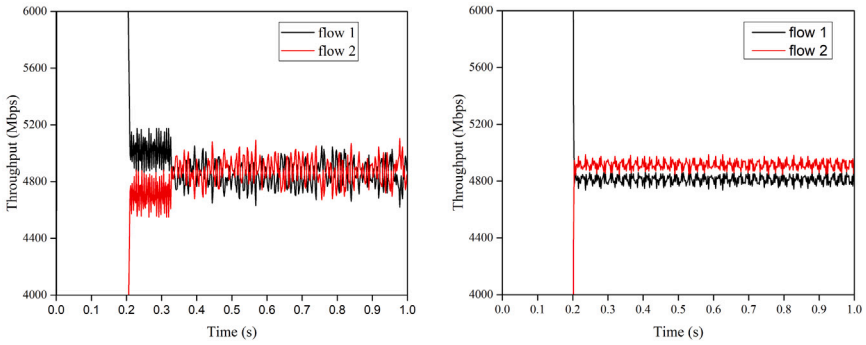
The results reveal that DCTCP's convergence speed is significantly lower than that of DC-Vegas. It takes DCTCP approximately 120 ms to reach convergence, whereas DC-Vegas requires only about 8 ms. However, we find that DC-Vegas can quickly converge to an unfair equilibrium, a phenomenon we will elucidate in the following section.

3.2. Cause analysis

In this subsection, we delve into the root cause of the unfairness and incorrect convergence exhibited by DC-Vegas.

In DC-Vegas, it calculates the current network queue length, denoted as Q_c, using Eq. (1) and then compares Q_c with a threshold, represented by K_{dcv}. Subsequently, DC-Vegas senders measure the proportion of packets for whose Q_c exceeds the threshold K_{dcv} within each RTT (as shown in Eq. (2)) to estimate the extent of network congestion with Eq. (3). Finally, the senders adjust the congestion window (cwnd) in accordance with the congestion level determined by Eq. (4).

$$Q_c = \frac{\text{cwnd}}{\text{RTT}_c} \times (\text{RTT}_c - \text{RTT}_{min}) \tag{1}$$



(a) Convergence for DCTCP (b) Convergence for DC-Vegas

Fig. 5. Convergence for DCTCP and DC-Vegas.

$$F_{dcv} = \frac{\text{number of ACKs with } Q_c > K_{\text{dcv}}}{\text{Total number of ACKs in a window}} \tag{2}$$

$$\alpha_{dcv} = (1-g) \times \alpha_{dcv} + g \times F_{dcv} \tag{3}$$

$$\text{cwnd} = \begin{cases} \text{cwnd} \times (1 - \frac{\alpha_{dcv}}{2}) & \text{If } F_{dcv} > 0 \\ \text{cwnd} + 1 & \text{If } F_{dcv} = 0 \end{cases} \tag{4}$$

where RTT_c represents the currently sampled RTT, and RTT_{min} signifies the minimum sampled RTT since the connection is established. The value $0 < g < 1$ is the weight given to this new sample.

From the above equations, we can see that certain variables, namely cwnd, RTT_c, and RTT_{min}, have a direct impact on the accuracy of the queue length (Q_c). The sender is able to calculate the value of cwnd and RTT_c upon receiving an ACK packet while the acquisition of RTT_{min} faces challenges. The original intention of RTT_{min} is the end-to-end delay in the absence of queuing. But in fact, this observed RTT_{min} represents the minimum RTT observed by the sender in previous iterations. The difference between expected RTT_{min} (i.e., the end-to-end delay without queuing) and observed RTT_{min} can result in unfairness and inaccurate convergence for DC-Vegas.

We examine both synchronous and asynchronous scenarios mentioned above. In the synchronous case, all senders start packet transmission simultaneously and increment their cwnd by one maximum segment size (MSS) upon receiving ACK packets. During this process, they calculate the corresponding RTTs. Since the network traffic load is typically low in such instances, there is a substantial probability that this RTT will represent the minimum RTT (RTT_{min}). Additionally, the RTTs for different senders may be nearly identical. Consequently, these senders should have a similar cwnd as dictated by Eqs. (1)–(4). As a result, in the synchronous case, each sender operates at a comparable rate. This characteristic allows DC-Vegas to achieve convergence towards a fair value.

In the asynchronous case, DC-Vegas fails to achieve convergence towards a fairness value. The reason is that subsequent flows experience an increased RTT as a result of queuing delays caused by preceding flows. Then, these later flows calculate a smaller Q_c and underestimate the congestion level, as outlined in Eqs. (1)–(3). As a result, they transmit at a faster rate than intended. Due to these aforementioned reasons, the later flow attains a higher throughput than the preceding flows, further exacerbating the unfairness in the system.

This unfairness convergence issue is not limited to DC-Vegas but also affects other delay-based congestion control protocols such as TCP-Vegas, TIMELY, and DX. For instance, TIMELY has been demonstrated that it has no fixed point of convergence, thus achieving unfairness [54]. Similarly, DX addresses its shortcomings through the utilization of a self-updated coefficient.

Based on the above discussion, in this paper, we seek to build an elegant method that ensures fast convergence and good fairness across diverse network topologies.

4. DCCS design

Our aim is to devise a method that can attain optimal performance in terms of high throughput and low end-to-end delay across various DCN scenarios while maintaining resilience against traffic bursts, ensuring rapid convergence, and fostering fairness among competing flows. To achieve these goals, this paper introduces a novel congestion control algorithm for datacenter TCP called Dual Congestion Control Signals (DCCS). DCCS leverages the advantages provided by two congestion signals (i.e., ECN and delay), for a comprehensive optimization of datacenter TCP performance.

4.1. Basic idea

DCCS design is informed by observed performance variations between ECN-based and delay-based congestion control methods across scenarios. Our experiments reveal that while the delay signal shows rapid convergence, it lacks reliability and is overly sensitive. Conversely, the ECN signal accurately detects congestion but suffers from slow convergence and struggles with multi-bottleneck congestion.

DCCS aims to merge the ECN and delay signals effectively. Specifically, it sets a pre-defined threshold for the delay signal to maintain low end-to-end delay in normal network conditions. Meanwhile, the ECN signal is utilized to alleviate incast congestion during burst traffic. Furthermore, an EAD signal is proposed to tackle severe congestion (i.e., multi-bottleneck congestion). Finally, we propose a drain signal to estimate minimum RTT accurately. This signal integration allows DCCS to balance responsiveness and accuracy in congestion detection and mitigation within DCNs.

Algorithm 1 describes the comprehensive process of DCCS, which comprises three primary modules: (1) ACK measurement module; (2) Dual congestion control signals management module; (3) Congestion window updates management module. Upon the arrival of an ACK packet at the sender, DCCS initiates the ACK management module (line 4). This module calculates the network queue length and keeps track of the number of packets marked with RTT and ECN indications separately. Subsequently, if all packets from the previous window have been acknowledged, DCCS proceeds to the dual congestion signal management module. In this module, the appropriate congestion signal is determined based on the prevailing conditions (line 6). Following that, the congestion window is updated accordingly (line 7). Lastly, the system parameters are reset.

As discussed previously, EAR, like DCCS, combines ECN and RTT signals for congestion control. However, DCCS differs significantly: Firstly, while EAR tends to be overly aggressive in slightly congested networks and conservative in severely congested networks, DCCS offers more precise congestion control. Secondly, EAR overlooks the fairness issue caused by RTT signals, whereas DCCS addresses this by incorporating the drain signal. Lastly, while EAR presents an initial concept without comprehensive analysis and experimentation, our work

Algorithm 1: DCCS Algorithm

```
   Input: ACK
1  Initialize: totalAck = 0, signal = 0 ;
2  foreach Arrived ACK do
3      totalAck = totalAck + ACK.size ;
       /* congestion signal management                    */
4      ACKManagement(ACK) ;
5      if received all the ACKs of last window then
           /* dual congestion signal management           */
6          signal = DCSManagement() ;
           /* congestion window management                */
7          windowUpdate(signal) ;
8          reSet() ;
9      end
10 end
```

thoroughly discusses DCCS's implementation details and evaluates its performance extensively.

In the following parts, we will provide a detailed explanation of the intricacies of our DCCS mechanism.

4.2. ACK management

DCCS employs the ACK management module, clearly depicted in Algorithm 2, to detect the ECN congestion signal and the delay congestion signal.

In the ACK management module, for each received ACK packet, we first check whether it contains an ECN bit. The presence of this bit indicates that the current queue length at the bottleneck switch exceeds the predefined threshold K_{ECN}, signaling network congestion. Consequently, we tally the number of packets marked with ECN, which will be utilized for ECN congestion control (Lines 2–5). Furthermore, we check whether this ACK will trigger the delay signal. To identify the congested packet for the delay signal, the sender relies on a threshold K_{nq}, serving as the equivalent of K_{ECN} in DCTCP. Specifically, when the sender receives an ACK packet, it will calculate the current RTT (called RTT_c) and estimate the current network queuing delay (NQ) using Eq. (1). If $NQ > K_{nq}$, it is recognized as a delay congestion signal, and the acknowledged bytes are recorded (Lines 6–11). At last, we also compute the average network queuing delay ($NQ_{avg} = NQ_{avg} + w \times (NQ - NQ_{avg})$, w is the weight used for exponentially averaging network queuing delay), which will be employed in the following section for the dual congestion signal algorithm.

Algorithm 2: ACK management algorithm

```
   Input: ACK
   Output: ackedBytesDelay, ackedBytesEcn
1  Initialize: NQ = 0 ;
   /* check whether has ECN bit                          */
2  isECN ← hasECN(ACK) ;
3  if isECN then
4      ackedBytesEcn+ = ACK.size;
5  end
   /* network queue and the avg value                    */
6  RTT_c ← getRTT(ACK) ;
7  updateMinRTT(RTT_c) ;
8  NQ ← cwnd × (RTT_c − RTT_min)/RTT_c ;
9  if NQ > K_nq then
10     ackedBytesDelay+ = ACK.size;
11 end
12 NQ_avg ← NQ_avg + w × (NQ − NQ_avg);
```

4.3. Dual congestion signals management

The key aspect of DCCS lies in its ability to smoothly transition congestion signals across various scenarios. In this subsection, we introduce a dual congestion signals-based management algorithm, referred to as DCSM, to facilitate the activation of different congestion signals, including delay signal, ECN signal, EAD signal, and drain signal.

Delay signal. The delay signal is triggered when a persistent queuing delay surpasses a predefined threshold but without ECN-marked ACK packets. This means there is no obvious network congestion but the accumulative queuing delay is high enough that affects network performance. The purpose of the delay signal is to mitigate this sustained queuing delay and maintain a low end-to-end network delay.

ECN signal. When ECN-marked ACK packets arrive at the sender, the ECN signal will take effect to control the burst traffic. This signal behaves in a manner similar to DCTCP.

EAD signal. When DCSM perceives ECN signal and the average network queuing delay exceeds a predefined threshold (K_m), it indicates severe network congestion occurring in multiple network bottlenecks. DCSM leverages the EAD signal to effectively tackle the traffic congestion while ensuring a low network delay.

Drain signal. As discussed in Section 3, the delay-based protocols often face fairness issues due to the inability to accurately determine the minimum RTT. To rectify this, we draw inspiration from the drain mechanism in BBR [45] and propose the integration of a drain signal within DCCS. The drain signal periodically drains the switch buffer, allowing for the accurate estimation of the minimum RTT.

To clarify, the details of DCSM is presented in Algorithm 3. First, DCSM calculates the fraction of ACK packets that are marked with ECN and delay separately (Lines 2–5). When the sender only observes the delayed ACK packets but without an ECN-marked packet, the delay signal is triggered (Lines 6–8). Subsequently, upon receiving ECN-marked ACK packets, the sender activates the ECN signal to manage congestion (Lines 9–11). Moreover, if the sender receives ECN-marked packets simultaneously the average network queuing delay is larger than a threshold value (K_m), it suggests severe network congestion occurring across multiple bottlenecks. To address this scenario, the EAD signal is generated (Lines 12–14). In addition, to ensure transmission fairness, DCSM incorporates a drain signal that periodically empties the switch buffer in order to accurately estimate the minimum RTT (Lines 15–19). A detailed explanation of this drain signal can be found in Section 4.5. These congestion signals are strictly switched based on different scenarios, each with its own priority ranking from low to high.

4.4. Congestion window update management

After obtaining the network congestion signal, it is essential to establish an appropriate congestion window update method according to different congestion signals. This approach aims to mitigate network congestion while preserving overall network performance.

Algorithm 4 describes the entire process of congestion window update. Among these signals, we have implemented standard procedures for two signals: the no congestion signal and the ECN signal. When there is no congestion signal, we use the additive increase mechanism, which increases the congestion window by one MSS during each RTT. On the other hand, if DCCS receives an ECN signal, it behaves similarly to the DCTCP protocol, and the congestion window is updated using the formula $cwnd = cwnd \times (1 - Alpha_{ECN}/2)$. These procedures are illustrated in lines 2–8 of Algorithm 4.

Next, we delve into the adjustment of the congestion window concerning delay, EAD, and drain signals. Regarding the delay signal, DCCS employs a conservative scheme to eliminate this delay while preserving network throughput. While we initially contemplate utilizing the TCP Vegas method, which advocates decreasing the congestion window by one MSS each RTT, we face a fairness issue among different flows

Algorithm 3: Dual congestion signals management algorithm

Input: ackedBytesDelay, ackedBytesEcn, NQ_{avg}, K_m
Output: $signal$
1 **Initialize:** $Alpha_{ECN} = 0, Alpha_d = 0, ECNFrac = 0, DelayFrac = 0, signal = non\_sig$;
2 $ECNFrac \leftarrow ackedBytesEcn/totalAck$;
3 $Alpha_{ECN} \leftarrow (1-g) \times Alpha_{ECN} + g \times ECNFrac$;
4 $DelayFrac \leftarrow ackedBytesDelay/totalAck$;
5 $Alpha_d \leftarrow (1-g) \times Alpha_d + g \times DelayFrac$;
6 **if** *DelayFrac* > 0 *AND ECNFrac* $== 0$ **then**
7 $signal \leftarrow delay\_sig$;
8 **end**
9 **if** $ECNFrac > 0$ **then**
10 $signal \leftarrow ecn\_sig$;
11 **end**
12 **if** $ECNFrac > 0$ *AND* $NQ_{avg} >= K_m$ **then**
13 $signal \leftarrow ead\_sig$;
14 **end**
15 $round+ = 1$;
16 **if** $round == drain\_cycle$ **then**
17 $signal \leftarrow drain\_sig$;
18 $round \leftarrow 1$;
19 **end**
20 **return** $signal$;

Algorithm 4: Window update algorithm

Input: $signal, cwnd_{in}$
Output: $cwnd$
1 $cwnd \leftarrow 0$;
2 **switch** $signal$ **do**
3 **case** non_sig **do**
4 $cwnd \leftarrow cwnd_{in} + 1$;
5 **end**
6 **case** ecn_sig **do**
7 $cwnd \leftarrow cwnd_{in} \times (1 - Alpha_{ECN}/2)$;
8 **end**
9 **case** $delay\_sig$ **do**
10 $cwnd \leftarrow cwnd_{in} - (1 - Alpha_d)$;
11 **end**
12 **case** ead_sig **do**
13 $Alpha_{EAD} = Alpha_{ECN} + Alpha_d$;
14 $cwnd \leftarrow cwnd_{in} \times (1 - Alpha_{EAD}/2)$;
15 **end**
16 **case** $drain\_sig$ **do**
17 $cwnd \leftarrow min(cwnd_{in}, drain\_cwnd)$;
18 **end**
19 **end**
20 **return** $cwnd$;

with variable minimum RTTs. Late arrivals may experience reduced congestion or no congestion at all due to their estimation of a smaller network delay resulting from larger minimum RTT estimates. Consequently, they might obtain higher throughputs than earlier flows. To compensate for this unfairness, we use Eq. (5) to regulate the congestion window (Lines 9–11 in Algorithm 4).

$$cwnd = cwnd - (1 - Alpha_d) \tag{5}$$

For the EAD signal, it indicates that the network suffers severe congestion. In this situation, we attempt to take aggressive action to adjust the congestion window. The sender calculates separate ECN congestion indicator and delay congestion indicator and then combines them to form a new congestion indicator, shown in Eq. (6). Subsequently, the congestion window is adjusted based on the well-established control laws of DCTCP, as shown in Eq. (7) (Lines 12–15).

$$Alpha_{EAD} = Alpha_{ECN} + Alpha_d \tag{6}$$

$$cwnd = cwnd \times (1 - Alpha_{EAD}/2) \tag{7}$$

When receiving the drain signal, the sender should decrease its congestion window to maintain a sufficiently low flow rate and avert network queuing. This recurring drain signal permits flows to perceive their minimum RTTs accurately, thereby promoting fairness. However, striking an optimal balance in decreasing the congestion window is crucial to avoid network under-utilization while successfully draining the queue. To achieve this, we suggest setting the congestion window to a predetermined value, referred to as $drain\_cwnd$, for a period of $drain\_cycle$ RTTs. This strategy empties the queue effectively without impairing network performance. We will explore suitable values for $drain\_cwnd$ and $drain\_cycle$ in the following section.

4.5. Parameters analysis

Our DCCS employs five crucial parameters: K_{ECN}, K_{nq}, K_m, $drain\_cycle$, and $drain\_cwnd$. Of these, we adopt the default values proposed in [1,10] for K_{ECN} and K_{nq}, respectively, and do not delve into them further in this paper. Instead, we focus on optimizing the settings of K_m, $drain\_cycle$, and $drain\_cwnd$.

Setting for K_m. K_m serves as a threshold to determine whether serious network congestion happens in the case of multi-bottleneck congestion. In general, a suitable choice for generating the EAD signal is when the instant network queue length exceeds twice the congested queuing lengths. However, in practical scenarios, we cannot obtain the instant network queue length because the DCSM module is triggered in every RTT period. Hence, we apply average network queue length (NQ_{avg}) to indicate the congestion level. Moreover, to accommodate burst traffic, we set $K_m = 1.5 \times K_{nq}$. As a result, the EAD signal is generated when $NQ_{avg} \geq K_m$ ($NQ_{avg} \geq 1.5 \times K_{nq}$). This enables the system to effectively tolerate and manage bursty network traffic.

Choice of $drain\_cwnd$ and $drain\_cycle$. To explore the precise minimum RTT, DCCS triggers a $drain\_signal$ every $drain\_cycle$ rounds and sets the congestion window to $drain\_cwnd$ to drain the network queue.

Firstly, let us discuss how to choose a suitable value for $drain\_cwnd$. It is crucial to ensure that with this particular $drain\_cwnd$, the network queue should not contain any packets. In other words, the aggregate throughput achieved by all flows should be less than the capacity of the bottleneck switch. This condition can be represented by the following Eq. (8):

$$\sum_{i=0}^{N} cwnd(i) \leq \mathrm{RTT} \times C \tag{8}$$

where we consider there are N flows with identical round-trip times (in seconds) with no queuing delay (called RTT), sharing a single bottleneck link of capacity C (in packets/second). The $cwnd(i)$ is the congestion window for flow i. We assume that all flows are synchronized, exhibiting the same behavior and sharing the same congestion window. From these assumptions, we can deduce the following relationship:

$$cwnd(i) \leq \frac{\mathrm{RTT} \times C}{N} \tag{9}$$

Since DCCS aims to rapidly improve network performance after receiving the drain signal, it is preferable to choose a value for the cwnd as large as possible. Therefore, the optimal value of cwnd can be obtained as follows:

$$cwnd(i) = \lfloor \frac{\mathrm{RTT} \times C}{N} \rfloor \tag{10}$$

Furthermore, considering the extreme case where $N = 1$, this implies that even with only one flow in the network, DCCS should

ensure its throughput. Building on the above discussion, the value of cwnd can be determined as follows:

$$cwnd(1) = \lfloor \mathrm{RTT} \times C \rfloor \tag{11}$$

Finally, we believe that cwnd should be larger than the initial window and then we get:

$$drain\_cwnd = max(\lfloor \mathrm{RTT} \times C \rfloor, W_{init}) \tag{12}$$

where W_{init} is the initial window size for our TCP.

Secondly, we discuss the parameter $drain\_cycle$. It is essential to strike a balance when setting the value of $drain\_cycle$ as it directly affects the behavior of the congestion window updates. If $drain\_cycle$ is too small, frequent reductions in the congestion window may occur, leading to a degradation in network throughput. On the other hand, if $drain\_cycle$ is set with a large value, there is a higher probability that the mice flows may not be able to probe a correct minimum RTT. This is because the FCT of mice flows typically spans only tens of RTTs in DCNs. Therefore, it is crucial to carefully choose a suitable $drain\_cycle$ that maintains a good throughput while accurately obtaining the minimum RTT. Furthermore, the congestion window increases by one MSS in each RTT in the absence of congestion signals. We assume that it can achieve the maximum network throughput after D RTTs without any congestion signal. We can express this as follows:

$$\sum_{i=0}^{N} (cwnd(i) + D) \leq \mathrm{RTT} \times C + B_{\Delta} \tag{13}$$

where B_{Δ} is a parameter that represents when buffer occupancy reaches the size of B_{Δ}, no congestion signal is triggered. We also assume all the flows are synchronized and each flow has the same behavior, then we get:

$$D \leq \frac{\mathrm{RTT} \times C + B_{\Delta}}{N} - cwnd(i) \tag{14}$$

Now we should find a maximum value for D in the different situations with variable N and $cwnd(i)$. Obviously, when $N = 1$ and $cwnd(i) = W_{init}$, we can get a maximum value for D ($D = \mathrm{RTT} \times C + B_{\Delta} - W_{init}$). This means that we should guarantee that even if there is only one flow, this flow can fully utilize the network bandwidth after D RTTs. After that, we let $drain\_cycle$ be equal to this maximum D, so we get

$$drain\_cycle = \mathrm{RTT} \times C + B_{\Delta} - W_{init} \tag{15}$$

Finally, for the situation with multiple congestion points, we should choose the minimum value of $drain\_cwnd$ and $drain\_cycle$ along the path.

Practical considerations. To calculate the values of $drain\_cwnd$ and $drain\_cycle$, we need to obtain the following parameters: RTT, C, W_{init}, and B_{Δ}, as indicated by Eqs. (12) and (15). Among these parameters, C represents the network bandwidth of the congested link. In DCNs, network congestion primarily occurs at the edge switches. Moreover, the link bandwidth between server and the ToR switch will not exceed that between the switches. Hence, we consider the link bandwidth between the server and the ToR switch as the value of C, which can be easily accessible by the sender. W_{init} is determined within the protocol stack. For our experiments, we set $W_{init} = 10$. The value of B_{Δ} should guarantee no congestion signal occurs, i.e., ECN signal and delay signal. Therefore, on one side, we should keep B_{Δ} smaller than K_{ECN} to ensure no ECN signal generated. On the other side, B_{Δ} should be small enough to maintain the Q_c calculated by sender be no larger than K_{nq}. As we know, $K_{nq} < K_{ECN}$, for the sake of simplicity, we set $B_{\Delta} = K_{nq}$.

Obtaining the RTT value is slightly more complex. Theoretically, the RTT value should be measured under uncongested network conditions. However, in practice, when flows are generated by the sender, the current flow's RTT value is obtained by receiving ACK packets. It is possible that network congestion may occur during flow generation, leading to inaccurate RTT values. Nevertheless, we can still calculate $drain\_cwnd$ and $drain\_cycle$ using Eqs. (12) and (15). During the $drain\_cycle$ period, the RTT value is continuously monitored to identify the minimum RTT value. Once the $drain\_cycle$ period is reached, this observed minimum RTT is used to accurately calculate the values of $drain\_cwnd$ and $drain\_cycle$. Afterwards, the $drain\_cwnd$ and $drain\_cycle$ values for this flow will no longer be updated.

Finally, since Eq. (12) is derived under ideal conditions without considering the burst characteristics of DCN traffic, setting the $drain\_cwnd$ value in practical systems must account for flow burstiness. In our simulation experiments, we set the value of $drain\_cwnd$ to 60% of the calculated result from Eq. (12).

5. Evaluation

In this section, we employ ns-3 simulations [55] to address the following three questions:

1. What are the DCCS behaviors exhibited under different scenarios in DCNs?
2. Is it possible for DCCS to rapidly converge to fairness?
3. How does DCCS perform in large-scale networks with realistic workloads?

We initiate by analyzing DCCS's fundamental properties in a simple network topology, focusing its influence over crucial performance metrics: throughput, flow completion time (FCT), queue length, convergence, and fairness. Moving to a large-scale network topology, representative of two realistic DCN workloads, we aim to highlight DCCS's efficiency, particularly in terms of FCT, which is a crucial metric in DCNs. For robustness and reliability, we conduct simulations ten times with different randomly selected start times, averaging the results from each run unless specified otherwise.

5.1. Parameter settings

Our simulations are running under two distinct network topologies including simple and large-scale scenarios shown in Figs. 1 and 3.

All link speeds are set at 10 Gbps, and to maintain consistency with this shallow buffer, the switch in our simulation employs a buffer of 250 packets per port, amounting to approximately 375 kB (250×1.5 kB).

DCCS uses both ECN and delay signals as means to alleviate network congestion. To establish meaningful benchmarks for our evaluation, we consider DCTCP and DC-Vegas as representative ECN-based and Delay-based methods, respectively. To ensure fair comparisons, we adopt the default parameter settings recommended in [1] for DCTCP and in [10] for DC-Vegas. These parameter configurations have been widely accepted within the research community and are considered standard for evaluating the performance of these protocols. In addition, we also include Swift [39] and EAR [44] in our comparative analysis. Swift represents the most recent delay-based protocol, while EAR is a mixed signal protocol combining ECN and delay signals. The values of $drain\_cwnd$ and $drain\_cycle$ can be determined by Eqs. (12), (15) and practical consideration. For all other parameters used in our simulations, unless stated otherwise, their values can be found in Table 2.

5.2. Simple scenario

In the context of this simple scenario, we employ the network topology illustrated in Fig. 3. Unless specified otherwise, the flow quantities of TR 1 and TR 2 are set to 30 and 50, respectively.

Incast throughput. We evaluate the performance of DCCS under the incast scenario, using a variety of flow number for TR 3 ranging from 10 to 300 with a fixed size of 1 MB per flow. We calculate the average throughput and the result is shown in Fig. 6. From this figure, DCCS maintains consistent performance even when faced with about

Table 2
Parameters setting for DCCS.

Parameters	Value
10 Gbps Link Delay	10 μs
K_{ECN} for 10 Gbps network	50
K_{nq} for 10 Gbps network	16
B_{Δ}	16
W_{init}	10
RTO_{min}	10 ms
Packets size	1500 Byte

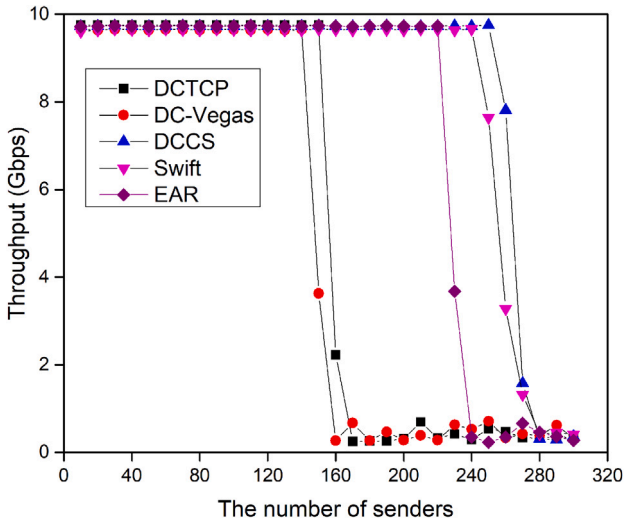


Fig. 6. Incast throughput with varying flow number of TR 3.

260 concurrent flows, whereas DCTCP, DC-Vegas, Swift, and EAR experience noticeable performance degradation when the number of flows exceeds 155, 140, 250, and 230, respectively. This finding suggests that DCCS is better equipped to handle burst traffic and demonstrates its ability to scale more effectively than other protocols.

Queue length. We fix the flow number of TR3 to 100 and initiate all three traffic simultaneously. After one second, we record data to examine the queuing behavior. Notably, there are two bottleneck ports involved, namely B1 and B2. While TR1 and TR2 contribute to bottleneck port B1, TR3 primarily causes congestion at bottleneck port B2. To analyze the queuing dynamics, we quantify the instantaneous queue lengths on both B1 and B2.

Fig. 7 shows the cumulative distribution function (CDF) of instantaneous queue lengths for both bottleneck ports, B1 and B2. Our examination of these figures reveal three key findings: Firstly, the delay-based methods (DC-Vegas, Swift, and DCCS) demonstrate a significantly lower queue length compared to the ECN-based approach (DCTCP) in a multi-hop and multi-bottleneck setting. As depicted in Fig. 7(a), the queue lengths for DCCS and Swift remain comparatively low and the 99th percentile is 115 and 125 packets. DC-Vegas and EAR gain a moderate queue length with the 99th percentile is 155 packets while DCTCP experiences highest queue length due to its reliance on ECN. Secondly, in a single-hop scenario, both DCCS and Swift achieve comparable lowest queue length while DC-Vegas suffers highest queue length, as illustrated in Fig. 7(b). This observation supports our previous analysis that the ECN-based method can mitigate incast congestion in a single-hop context. Finally, our proposed DCCS algorithm performs admirably in both multi-hop and multi-bottleneck situations owing to its hybrid design incorporating ECN and delay signals. Specifically, the average queue length of DCCS at B1 is approximately 88 packets, representing a reduction of 46%, 23.5%, 5.4%, and 26.7% relative to DCTCP, DC-Vegas, Swift, and EAR, respectively. Similarly, the average queue length of DCCS at B2 is around 60 packets, constituting a decrease of 16.7%, 20.4%, 4.8%, and 11.8% compared to DCTCP, DC-Vegas, Swift, and EAR.

Table 3
Jain's fairness index in asynchronous traffic.

	DCTCP	DC-Vegas	DCCS	Swift	EAR
TR1	0.989	0.672	0.995	0.998	0.706
TR2	0.992	0.771	0.996	0.998	0.817
TR3	0.991	0.724	0.993	0.996	0.763

Fairness. To evaluate the fairness of our DCCS, we start TR1, TR2, and TR3 asynchronously within a 0.5-s window, and subsequently, we collect data and compute the average fairness using Jain's Fairness Index [53] between 1.5 and 2 s. The Jain's Fairness Index values are listed in Table 3; a higher index denotes increased fairness. Our findings indicate that DCTCP exhibits satisfactory fairness, whereas DC-Vegas and EAR experience poor fairness. Moreover, our proposed DCCS and Swift algorithm achieve the highest fairness among all protocols, which indicates that our DCCS can guarantee good fairness.

Convergence. In order to thoroughly investigate the convergence properties of DCCS, we build two simulations to compare the convergence speed and fairness of DCCS in comparison to DCTCP, DC-Vegas, Swift, and EAR. As the convergence behavior of DCTCP and DC-Vegas has already been demonstrated in Fig. 5, our focus lies solely on the evaluation of DCCS, Swift, and EAR.

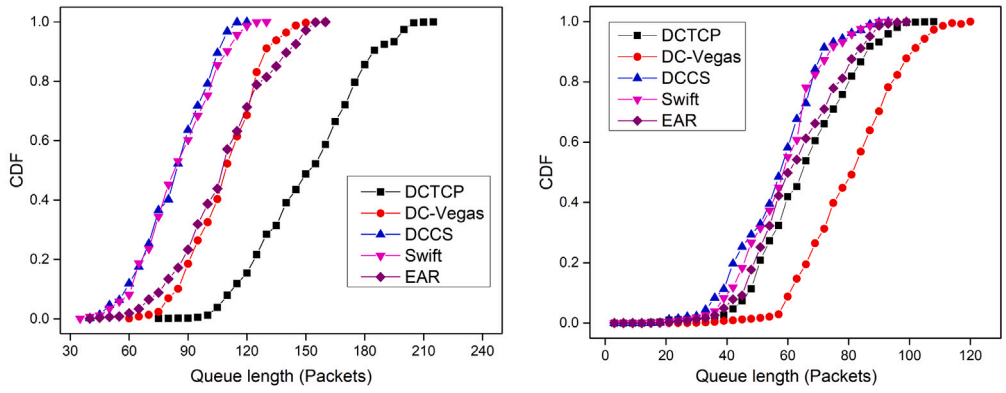
We initiate two flows scenarios in a 10 Gbps link, where one flow (flow 1) instantaneously captures the full link bandwidth upon activation, and a subsequent flow (flow 2) enters the network after a certain delay. We then observe the convergence behavior of distinct protocols under these conditions. The results are illustrated in Figs. 8(a), 8(b), and 8(c). The analysis reveals that EAR rapidly converges (within 6 ms) to an unfair sharing scheme, indicative of its bias towards high-priority traffic. DCCS demonstrates a remarkably convergence pace (approximately 6 ms), yet maintains modest fluctuations around its fair share. Meanwhile, Swift requires about 80 ms to converge as it needs dynamic adjustments of its target delay to reach its optimum value, although it would achieve better convergence stability and fairness.

Secondly, we generate 5 identical flows in the 10 Gbps link, each with a unique duration (27 s, 21 s, 15 s, 9 s, and 3 s) separated by 3-second intervals. The resulting convergence patterns are displayed in Figs. 8(d), 8(e), and 8(f). Our findings indicate that both DCCS and Swift achieve quick convergence to their optimal equilibria. DCCS leverages the strengths of both DCTCP and DC-Vegas, exhibiting rapid convergence like DC-Vegas while ensuring fair sharing like DCTCP. Swift also achieves quick fairness since it leverages the target delay for adjusting its congestion window without estimating the minimum RTT. However, EAR's convergence is found to be incorrect and hasty, leading to suboptimal performance.

5.3. Large-scale scenario

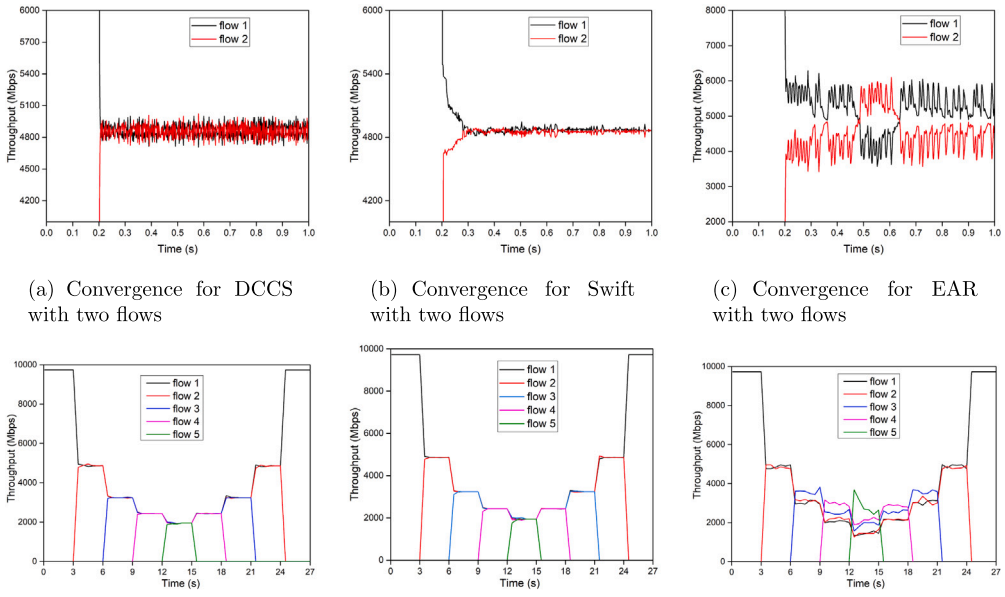
To further understand the performance of DCCS in a large-scale DCN environment, we conduct simulations using realistic network topologies and traffic workloads.

Topology. To evaluate the performance of DCCS in a more realistic setting, we employ a three-tier fat-tree topology with 8 Pods, as depicted in Fig. 1(c). Each Pod comprises a 4 × 4 leaf-spine architecture, where each leaf switch (also referred to as ToR switch) interconnects with 8 servers, forming a single-rack topology. Our simulation setup includes 16 core switches, 32 aggregation switches, and 32 ToR switches. Each ToR switch has 12 ports, with 8 ports connected to 8 servers, and the remaining 4 ports connected to 4 aggregation switches in the same Pod. Additionally, the aggregation switch has 8 ports, with 4 ports connected to the ToR switches and another 4 ports connected to the Core switch. All links are established at 10 Gbps with a latency



(a) CDF of Queue length at B1 (b) CDF of Queue length at B2

Fig. 7. The queue length at switch bottlenecks B1 and B2.



(a) Convergence for DCCS with two flows (b) Convergence for Swift with two flows (c) Convergence for EAR with two flows

(d) Convergence for DCCS with five flows (e) Convergence for Swift with five flows (f) Convergence for EAR with five flows

Fig. 8. Convergence for DCTCP, Swift, and EAR.

of 10 μs. Note that each ToR switch services 8 servers, yielding an oversubscription ratio of 2:1 at this layer.

Workload. In our simulation, we utilize two distinct real data center workloads: the web search workload [1] and the data mining workload [56]. For both workloads, we generate 2000 flows according to a Poisson arrival process, with flow sizes ranging from 10 kB to 50 MB. Approximately 70% of flows in the web search workload are less than 1 MB in size, with over half falling between 100 kB and 1 MB. In contrast, more than 90% of the flows in the data mining workload are below 1 MB, with an impressive 95% of the total bytes attributable to the mere 5% of large flows exceeding 1 MB. It is worth noting that the flows within our simulation can be categorized into one of three traffic types based on their topology: single-rack traffic, leaf-spine traffic, and fat-tree traffic. Specifically, 20% of the flows are confined within the single-rack topology, 40% are contained within the leaf-spine topology, and the remaining 40% are distributed across the fat-tree topology.

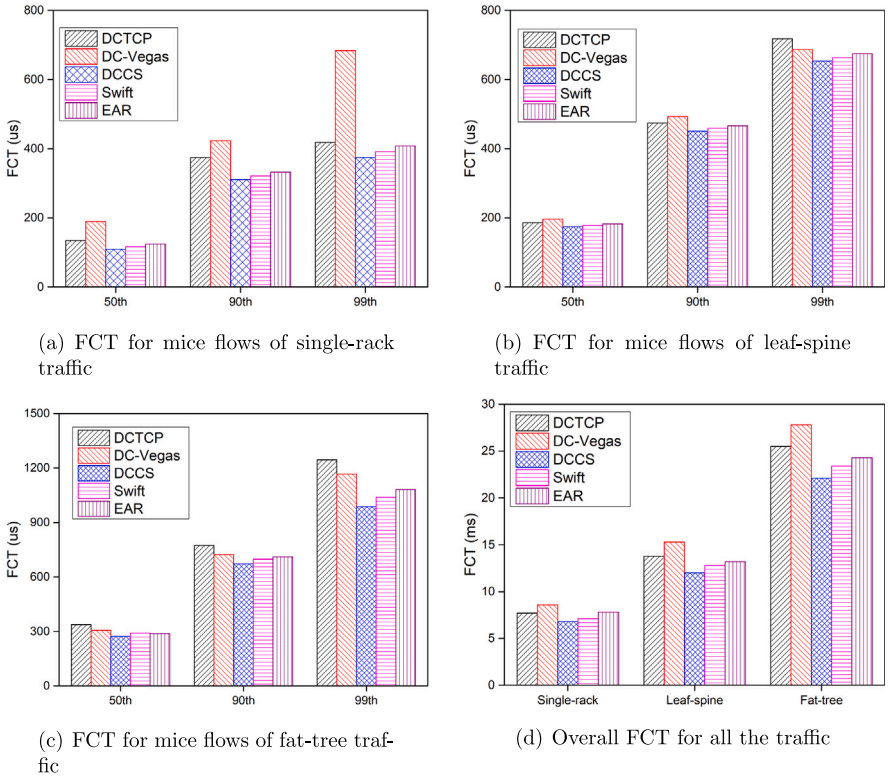
Metric. As the primary performance metric in DCNs, we utilize four metrics to evaluate our results: 50th percentile FCT, 90th percentile FCT, 99th percentile FCT, and the average FCT of all the flows (called overall FCT). These metrics provide a comprehensive understanding of the network's performance under various load conditions.

Performance with web search workload. We first run a web search workload and record the FCTs of mice flows (less than 1 MB) for different traffic. The results are displayed in Figs. 9(a), 9(b), 9(c).

For the mice flows of single-rack traffic, the FCT of DCCS is lowest compared to DCTCP, DC-Vegas, Swift, and EAR, as depicted in Fig. 9(a). Notably, the 99th percentile FCT of DCCS is approximately 10.5%, 45.3%, 4.3%, and 8.3% lower than that of DCTCP, DC-Vegas, Swift, and EAR. Moreover, DC-Vegas exhibits highest FCT. This observation can be attributed to the fact that delay-based protocols struggle to manage bursty flows in the single-hop scenario, whereas ECN-based protocols can perform effectively in such scenarios.

As the network topology grows larger and more complex, DCTCP exhibits inferior performance relative to other protocols, shown in Figs. 9(b) and 9(c), while DCCS achieves the lowest FCT across all percentiles, including the 50th percentile FCT, 90th percentile FCT, and 99th percentile FCT. This observed difference in performance can be attributed to DCCS's innovative dual congestion control signal mechanism, allowing it to better manage network congestion and mitigate the negative impacts of bursty traffic.

In addition, we plot the overall FCT of all flows in Fig. 9(d). The results show that DCCS also has the lowest FCT among these protocols.



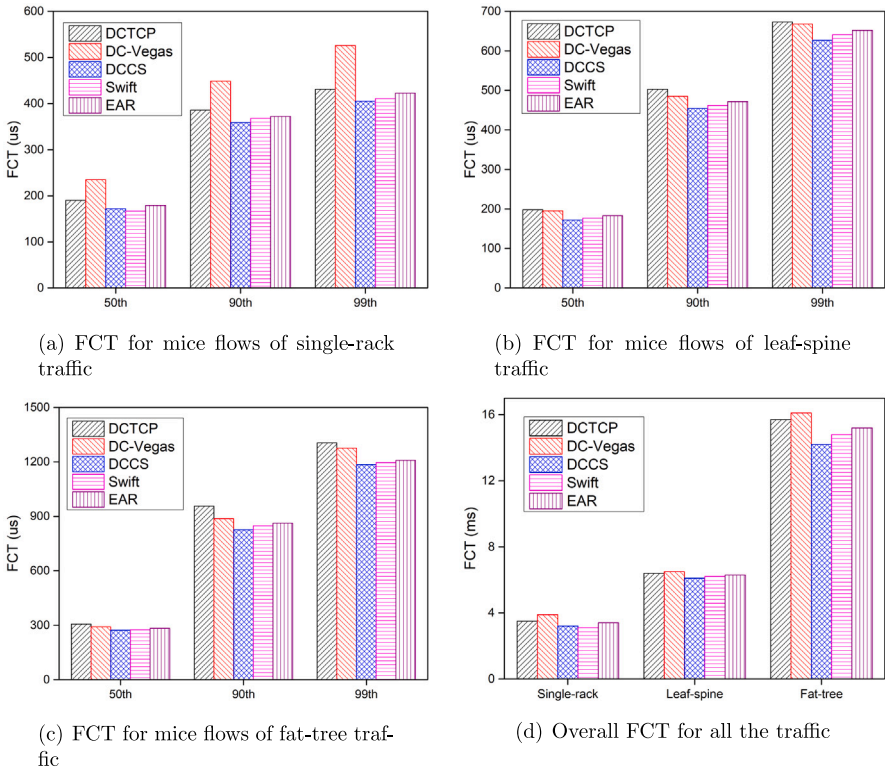
(a) FCT for mice flows of single-rack traffic

(b) FCT for mice flows of leaf-spine traffic

(c) FCT for mice flows of fat-tree traffic

(d) Overall FCT for all the traffic

Fig. 9. The FCT for web search workload.



(a) FCT for mice flows of single-rack traffic

(b) FCT for mice flows of leaf-spine traffic

(c) FCT for mice flows of fat-tree traffic

(d) Overall FCT for all the traffic

Fig. 10. The FCT for data mining workload.

Specifically, in the single-rack and leaf-spine topologies, the FCT of DCCS is just slightly smaller, but in the fat-tree topology, DCCS reduces the overall FCT by up to 13.3%, 20.5%, 5.6%, and 9.1% compared to DCTCP, DC-Vegas, Swift, and EAR.

Performance with data mining workload. We further examine the FCT with a data mining workload, as shown in Fig. 10. Our findings mirror those of the web search workload, demonstrating that DCCS consistently outperforms DCTCP, DC-Vegas, Swift, and EAR across various

traffic scenarios. More specifically, we observe that for flows under fat-tree traffic, DCCS achieves a significantly lower 99th percentile FCT compared to DCTCP, DC-Vegas, Swift, and EAR, with improvements ranging from up to ~11.8%, ~9.8%, ~3.8%, and ~4.7%, respectively.

In conclusion, our evaluations demonstrate that DCCS outperforms its competitors across all workloads due to its ability to promptly respond to burst congestion through the use of ECN signals, while simultaneously maintaining low end-to-end delays via delay signals.

6. Conclusions

To achieve high throughput and low end-to-end delay in DCNs, we investigate the integration of ECN and delay-based congestion control mechanisms. Moreover, we propose a novel mixed signal approach called EAD that combines the benefits of both techniques to effectively tackle server congestion. Additionally, to ensure fairness among flows, we develop a drain signal to precisely determine the minimum RTT for each flow. Building upon these congestion control signals, we introduce the DCCS protocol. The synergistic effects of ECN and delay signals enable DCCS to offer several advantages over traditional TCP variants, including high throughput and robustness against burst traffic, low end-to-end delay and rapid convergence, and fairness ensured by the drain signal. Our extensive experimental evaluations conducted in diverse network settings demonstrate that DCCS outperforms existing protocols like DCTCP, DC-Vegas, Swift. Finally, a large-scale simulated study utilizing a realistic workload confirms the superiority of DCCS over other protocols in DCNs. In future, we will continue to optimize the parameters of this paper.

CRediT authorship contribution statement

Yifei Lu: Writing – review & editing, Methodology, Conceptualization. **Xu Ma:** Writing – original draft, Validation, Software, Investigation. **Changjiang Cui:** Validation, Software, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgments

This research was supported by the National Natural Science Foundation of China under Grant No. 61702267.

References

[1] M. Alizadeh, A. Greenberg, D.a. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, M. Sridharan, Data center TCP (DCTCP), ACM SIGCOMM Comput. Commun. Rev. 40 (4) (2010) 63–74, http://dx.doi.org/10.1145/1851275.1851192.

[2] R. Mittal, V.T. Lam, N. Dukkipati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, D. Zats, TIMELY: RTT-based congestion control for the datacenter, SIGCOMM Comput. Commun. Rev. 45 (4) (2015) 537–550, http://dx.doi.org/10.1145/2829988.2787510.

[3] P. Cheng, F. Ren, R. Shu, C. Lin, Catch the whole lot in an action: Rapid precise packet loss notification in data center, in: Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation, NSDI'14, Seattle, WA, USA, 2014, pp. 17–28.

[4] Y. Lu, Z. Ling, S. Zhu, L. Tang, SDTCP: Towards datacenter TCP congestion control with SDN for IoT applications, Sensors 17 (1) (2017) 1–20, http://dx.doi.org/10.3390/s17010109.

[5] Y. Lu, C. Cui, X. Ma, Z. Ruan, A^{3}DCT: A cubic acceleration TCP for data center networks, J. Netw. Comput. Appl. 216 (2023) 103654, http://dx.doi.org/10.1016/j.jnca.2023.103654.

[6] C. Wilson, H. Ballani, T. Karagiannis, A. Rowtron, Better never than late: Meeting deadlines in datacenter networks, SIGCOMM Comput. Commun. Rev. 41 (4) (2011) 50–61, http://dx.doi.org/10.1145/2043164.2018443.

[7] Y. Lu, SED: An SDN-based explicit-deadline-aware TCP for cloud data center networks, Tsinghua Sci. Technol. 21 (5) (2016) 491–499.

[8] Y. Lu, X. Fan, L. Qian, Dynamic ECN marking threshold algorithm for TCP congestion control in data center networks, Comput. Commun. 129 (2018) 197–208, http://dx.doi.org/10.1016/j.comcom.2018.07.036.

[9] C. Lee, C. Park, K. Jang, S. Moon, D. Han, DX: Latency-based congestion control for datacenters, IEEE/ACM Trans. Netw. 25 (1) (2017) 335–348, http://dx.doi.org/10.1109/TNET.2016.2587286.

[10] J. Wang, J. Wen, C. Li, Z. Xiong, Y. Han, DC-vegas: A delay-based TCP congestion control algorithm for datacenter applications, J. Netw. Comput. Appl. 53 (2015) 103–114, http://dx.doi.org/10.1016/j.jnca.2015.03.010.

[11] Y. Lu, X. Ma, Z. Xu, FAMD: A flow-aware marking and delay-based TCP algorithm for datacenter networks, J. Netw. Comput. Appl. 174 (2021) 102912, http://dx.doi.org/10.1016/j.jnca.2020.102912.

[12] M. Allman, V. Paxson, E. Blanton, RFC5681: TCP Congestion Control, Technical Report, 2009, URL: https://tools.ietf.org/html/rfc5681.

[13] T. Henderson, S. Floyd, A. Gurtov, Y. Nishida, The NewReno Modification to TCP's Fast Recovery Algorithm, Technical Report, 2012, URL: https://tools.ietf.org/html/rfc3782.

[14] S. Ha, I. Rhee, L. Xu, CUBIC: A new TCP-friendly high-speed TCP variant, Oper. Syst. Rev. 42 (5) (2008) 64–74, http://dx.doi.org/10.1145/1400097.1400105.

[15] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. Haj, Y. Ming, Congestion control for large-scale RDMA deployments, in: The 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM, 2015, pp. 523–536, http://dx.doi.org/10.1145/2785956.2787484.

[16] L.S. Brakmo, L.L. Peterson, TCP vegas: End to end congestion avoidance on a global internet, IEEE J. Sel. Areas Commun. 13 (8) (2006) 1465–1480, http://dx.doi.org/10.1109/49.464716.

[17] D.X. Wei, C. Jin, S.H. Low, S. Hegde, FAST TCP: Motivation, architecture, algorithms, performance, IEEE/ACM Trans. Netw. 14 (6) (2006) 1246–1259, http://dx.doi.org/10.1109/TNET.2006.886335.

[18] B. Briscoe, A. Brunstrom, A. Petlund, D. Hayes, D. Ros, I.-J. Tsang, S. Gjessing, G. Fairhurst, C. Griwodz, M. Welzl, Reducing internet latency: A survey of techniques and their merits, IEEE Commun. Surv. Tutor. 18 (3) (2016) 2149–2196, http://dx.doi.org/10.1109/COMST.2014.2375213.

[19] F. Han, M. Wang, Y. Cui, Q. Li, R. Liang, Y. Liu, Y. Jiang, Future data center networking: From low latency to deterministic latency, IEEE Netw. 36 (1) (2022) 52–58, http://dx.doi.org/10.1109/MNET.102.2000622.

[20] S. Arslan, N. McKeown, Switches know the exact amount of congestion, in: Proceedings of the 2019 Workshop on Buffer Sizing, BS '19, Association for Computing Machinery, New York, NY, USA, 2019, http://dx.doi.org/10.1145/3375235.3375245.

[21] D. Nagle, D. Serenyi, A. Matthews, The panasas ActiveScale storage cluster: Delivering scalable high bandwidth storage, in: Proceedings of the 2004 ACM/IEEE Conference on Supercomputing, SC '04, IEEE Computer Society, Washington, DC, USA, 2004, p. 53, http://dx.doi.org/10.1109/SC.2004.57.

[22] A. Phanishayee, E. Krevat, V. Vasudevan, D.G. Andersen, G.R. Ganger, G.A. Gibson, S. Seshan, Measurement and analysis of TCP throughput collapse in cluster-based storage systems, in: Proceedings of the 6th USENIX Conference on File and Storage Technologies, FAST '08, USENIX Association, Berkeley, CA, USA, 2008, pp. 12:1–12:14.

[23] V. Vasudevan, A. Phanishayee, H. Shah, E. Krevat, D.G. Andersen, G.R. Ganger, G.a. Gibson, B. Mueller, Safe and effective fine-grained TCP retransmissions for datacenter communication, ACM SIGCOMM Comput. Commun. Rev. 39 (4) (2009) 303, http://dx.doi.org/10.1145/1594977.1592604.

[24] D. Katabi, M. Handley, C. Rohrs, Congestion control for high bandwidth-delay product networks, SIGCOMM Comput. Commun. Rev. 32 (4) (2002) 89–102, http://dx.doi.org/10.1145/964725.633035.

[25] J. Zhang, F. Ren, X. Yue, R. Shu, C. Lin, Sharing bandwidth by allocating switch buffer in data center networks, IEEE J. Sel. Areas Commun. 32 (1) (2014) 39–51, http://dx.doi.org/10.1109/JSAC.2014.140105.

[26] D. Han, R. Grandl, A. Akella, S. Seshan, FCP: A flexible transport framework for accommodating diversity, ACM SIGCOMM Comput. Commun. Rev. 43 (4) (2013) 135–146, http://dx.doi.org/10.1145/2534169.2486004.

[27] Y. Li, R. Miao, H.H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, M. Yu, HPCC: High precision congestion control, in: Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 44–58, http://dx.doi.org/10.1145/3341302.3342085.

[28] S. Arslan, Y. Li, G. Kumar, N. Dukkipati, Bolt: Sub-RTT congestion control for ultra-low latency, in: 20th USENIX Symposium on Networked Systems Design and Implementation, NSDI 23, USENIX Association, Boston, MA, 2023, pp. 219–236, URL: https://www.usenix.org/conference/nsdi23/presentation/arslan.

[29] B. Vamanan, J. Hasan, T. Vijaykumar, Deadline-aware datacenter TCP (D^{2}TCP), in: Proceedings of the ACM SIGCOMM 2012 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '12, ACM, New York, NY, USA, 2012, pp. 115–126, http://dx.doi.org/10.1145/2342356.2342388.

[30] C. Zhou, W. Wu, D. Yang, T. Huang, L. Guo, B. Yu, Deadline and priority-aware congestion control for delay-sensitive multimedia streaming, in: Proceedings of the 29th ACM International Conference on Multimedia, MM '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 4740–4744, http://dx.doi.org/10.1145/3474085.3479208.
[31] J. Zhang, H. Shi, Y. Cui, F. Qian, W. Wang, K. Zheng, J. Wu, To punctuality and beyond: Meeting application deadlines with DTP, in: 2022 IEEE 30th International Conference on Network Protocols, ICNP, 2022, pp. 1–11, http://dx.doi.org/10.1109/ICNP55882.2022.9940391.
[32] W. Chen, P. Cheng, F. Ren, R. Shu, C. Lin, Ease the queue oscillation: Analysis and enhancement of DCTCP, in: 2013 IEEE 33rd International Conference on Distributed Computing Systems, 2013, pp. 450–459, http://dx.doi.org/10.1109/ICDCS.2013.22.
[33] Y. Lu, X. Ma, Z. Xu, Choose a correct marking position: ECN should be freed from tail mark, Comput. Netw. 197 (2021) 108329, http://dx.doi.org/10.1016/j.comnet.2021.108329.
[34] N. Khademi, G. Armitage, M. Welzl, S. Zander, G. Fairhurst, D. Ros, Alternative backoff: Achieving low latency and high throughput with ECN and AQM, in: 2017 IFIP Networking Conference (IFIP Networking) and Workshops, 2017, pp. 1–9, http://dx.doi.org/10.23919/IFIPNetworking.2017.8264863.
[35] D. Shan, F. Ren, Improving ECN marking scheme with micro-burst traffic in data center networks, in: IEEE INFOCOM 2017 - IEEE Conference on Computer Communications, 2017, pp. 1–9, http://dx.doi.org/10.1109/INFOCOM.2017.8057181.
[36] L.A. Grieco, S. Mascolo, Performance evaluation and comparison of westwood+, new reno, and Vegas TCP congestion control, SIGCOMM Comput. Commun. Rev. 34 (2) (2004) 25–38, http://dx.doi.org/10.1145/997150.997155.
[37] M. Hock, F. Neumeister, M. Zitterbart, R. Bless, TCP LoLa: Congestion control for low latencies and high throughput, in: 2017 IEEE 42nd Conference on Local Computer Networks, LCN, 2017, pp. 215–218, http://dx.doi.org/10.1109/LCN.2017.42.
[38] G. Carlucci, L. De Cicco, S. Holmer, S. Mascolo, Congestion control for web real-time communication, IEEE/ACM Trans. Netw. 25 (5) (2017) 2629–2642, http://dx.doi.org/10.1109/TNET.2017.2703615.
[39] G. Kumar, N. Dukkipati, K. Jang, H.M.G. Wassel, X. Wu, B. Montazeri, Y. Wang, K. Springborn, C. Alfeld, M. Ryan, D. Wetherall, A. Vahdat, Swift: Delay is simple and effective for congestion control in the datacenter, in: Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 514–528, http://dx.doi.org/10.1145/3387514.3406591.
[40] R. King, R. Baraniuk, R. Riedi, TCP-Africa: an adaptive and fair rapid increase rule for scalable TCP, in: Proceedings IEEE 24th Annual Joint Conference of the IEEE Computer and Communications Societies, Vol. 3, 2005, pp. 1838–1848, http://dx.doi.org/10.1109/INFCOM.2005.1498463.
[41] K. Tan, J. Song, Q. Zhang, M. Sridharan, A compound TCP approach for high-speed and long distance networks, in: Proceedings IEEE INFOCOM 2006. 25TH IEEE International Conference on Computer Communications, 2006, pp. 1–12, http://dx.doi.org/10.1109/INFOCOM.2006.188.
[42] A. Baiocchi, A.P. Castellani, F. Vacirca, YeAH-TCP: yet another highspeed TCP, in: Proc. PFLDnet, Vol. 7, 2007, pp. 37–42.
[43] P. Goyal, A. Narayan, F. Cangialosi, D. Raghavan, S. Narayana, M. Alizadeh, H. Balakrishnan, Elasticity detection: A building block for delay-sensitive congestion control, in: Proceedings of the Applied Networking Research Workshop, ANRW '18, ACM, New York, NY, USA, 2018, p. 75, http://dx.doi.org/10.1145/3232755.3232772.
[44] G. Zeng, W. Bai, G. Chen, K. Chen, D. Han, Y. Zhu, Combining ECN and RTT for datacenter transport, in: Proceedings of the First Asia-Pacific Workshop on Networking, APNet '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 36–42, http://dx.doi.org/10.1145/3106989.3107002.
[45] N. Cardwell, Y. Cheng, C.S. Gunn, S.H. Yeganeh, V. Jacobson, BBR: Congestion-based congestion control, Queue 14 (5) (2016) 50:20–50:53, http://dx.doi.org/10.1145/3012426.3022184.
[46] T. Zhang, J. Huang, J. Wang, J. Chen, Y. Pan, G. Min, Designing fast and friendly TCP to fit high speed data center networks, in: 2018 IEEE 38th International Conference on Distributed Computing Systems, ICDCS, 2018, pp. 43–53, http://dx.doi.org/10.1109/ICDCS.2018.00015.
[47] K. Winstein, H. Balakrishnan, TCP ex machina: Computer-generated congestion control, SIGCOMM Comput. Commun. Rev. 43 (4) (2013) 123–134, http://dx.doi.org/10.1145/2534169.2486020.
[48] M. Dong, Q. Li, D. Zarchy, P.B. Godfrey, M. Schapira, PCC: Re-architecting congestion control for consistent high performance, in: 12th USENIX Symposium on Networked Systems Design and Implementation, NSDI 15, USENIX Association, Oakland, CA, 2015, pp. 395–408.
[49] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, M. Schapira, PCC vivace: Online-learning congestion control, in: 15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 18, USENIX Association, Renton, WA, 2018, pp. 343–356.
[50] W. Li, S. Gao, X. Li, Y. Xu, S. Lu, TCP-NeuRoc: Neural adaptive TCP congestion control with online changepoint detection, IEEE J. Sel. Areas Commun. 39 (8) (2021) 2461–2475, http://dx.doi.org/10.1109/JSAC.2021.3087247.
[51] J. Zhou, X. Qiu, Z. Li, Q. Li, G. Tyson, J. Duan, Y. Wang, H. Pan, Q. Wu, A machine learning-based framework for dynamic selection of congestion control algorithms, IEEE/ACM Trans. Netw. 31 (4) (2023) 1566–1581, http://dx.doi.org/10.1109/TNET.2022.3220225.
[52] H. Jiang, Q. Li, Y. Jiang, G. Shen, R. Sinnott, C. Tian, M. Xu, When machine learning meets congestion control: A survey and comparison, Comput. Netw. 192 (2021) 108033, http://dx.doi.org/10.1016/j.comnet.2021.108033.
[53] R. Jain, D.-M. Chiu, W. Hawe, A Quantitative Measure Of Fairness And Discrimination For Resource Allocation In Shared Computer Systems, DEC Research Report TR-301, Sep. 1984.
[54] Y. Zhu, M. Ghobadi, V. Misra, J. Padhye, ECN or delay: Lessons learnt from analysis of DCQCN and TIMELY, in: Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies, CoNEXT '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 313–327, http://dx.doi.org/10.1145/2999572.2999593.
[55] ns3, 2011. https://www.nsnam.org/, [Online; accessed 3-May-2023].
[56] A. Greenberg, J.R. Hamilton, N. Jain, S. Kandula, C. Kim, P. Lahiri, D.A. Maltz, P. Patel, S. Sengupta, VL2: A scalable and flexible data center network, in: Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication, SIGCOMM '09, Association for Computing Machinery, New York, NY, USA, 2009, pp. 51–62, http://dx.doi.org/10.1145/1592568.1592576.

Yifei Lu received his BS and Ph.D. degrees in Computer Sciences from Southeast University in 2005, 2010, respectively. He is currently an associate professor of the School of Computer Science and Engineering, Nanjing University of Science and Technology, China. His research interests include the Computer networks, Software defined networking, with a special focus on data center networks.

Xu Ma received the BS degree in Software Engineering from Jinling Institute of Technology. He is working towards the MS degree in computer science at Nanjing University of Science and Technology. His research interest includes TCP congestion control in datacenter networks.

Changjiang Cui received the BS degree in computer science at Nanjing University of Science and Technology. He is working towards the MS degree in computer science at Nanjing University of Science and Technology. His research interest includes QUIC protocol, TCP congestion control in datacenter networks.